

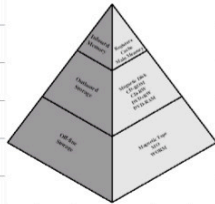
INTRODUZIONE

- IL RUOLO DELLA MEMORIA SECONDO VON NEUMANN $\Rightarrow$ UN SISTEMA DI VON NEUMANN HA 3 BLOCCHI FONDAMENTALI (UNITÀ): UNITÀ CENTRALE DI ELABORAZIONE (CPU), LA MEMORIA E L'UNITÀ I/O (INGRESSO/USCITA)
- LA MEMORIA CONSERVA DATI, ISTRUZIONI E TUTTO CIÒ CHE SERVE AL FUNZIONAMENTO DEL SISTEMA
- OGGI SI PARLA DI SISTEMA DI MEMORIA (O UNITÀ DI MEMORIA), PERCHÉ IN UN COMPUTER MODERNO LA "MEMORIA" È UN INSIEME GERARCHICO DI MODULI (CACHE, RAM, DISCHI, ECC...) CHE COLLABORANO FRA LORO
- CLASSIFICAZIONE ELEMENTARE DELLA MEMORIA

- $\rightarrow$ MEMORIA INTERNA: ACCESSIBILE DIRETTAMENTE DAL PROCESSORE. TIPICAMENTE I REGISTRI INTERNI, LE CACHE L₁/L₂/L₃ E LA RAM PRINCIPALE
- $\rightarrow$ MEMORIA ESTERNA: RICHIEDE UN'INTERFACCIA DI I/O PER ESSERE RAGGIUNTA DALLA CPU. TIPICAMENTE DISCHI MAGNETICI, SSD, NASTRI, ARCHIVI DI RETE

PERCHÉ UNA GERARCHIA DI MEMORIA? NESSUNA TECNOLOGIA OFFRE SIMULTANEAMENTE IL BASSO COSTO PER BIT, ALTA CAPACITÀ E TEMPI DI ACCESSO RIDOTTI

GERARCHIE DI MEMORIA



- LIVELLO PIÙ ALTO (VICINO ALLA CPU)
 - $\rightarrow$ REGISTRI INTERNI: POCHISSIMI BYTE, ACCESSO ISTANTANEO
 - $\rightarrow$ CACHE (L₁, L₂, L₃)
- MEMORIA PRIMARIA: DRAM (RAM PRINCIPALE)
- MEMORIA SECONDARIA (ESTERNA): SSD, HDD, CD/DVD, Blu-Ray
- STORAGE OFFLINE: TAPE, WORM, MO. ACCESSO SEQUENZIALE, USATO PER BACKUP A LUNGO TERMINE

PERCHÉ QUESTO SCHEMA?

NON ESISTE UN'UNICA TECNOLOGIA CHE CONIUGHI BASSO COSTO, ALTA CAPACITÀ E BASSA LATENZA, QUINDI SI COMPONE UN SISTEMA A LIVELLI

- PIÙ "VICINO" $\rightarrow$ VELOCE MA COSTOSO E PICCOLO
- PIÙ "LONTANO" $\rightarrow$ LENTO MA ECONOMICO E CAPIENTE

CARATTERISTICHE DI UN SISTEMA DI MEMORIA

I DIVERSI TIPI DI MEMORIA VENGONO CLASSIFICATI SULLA BASE DELLE SEGUENTI CARATTERISTICHE:

Location COLLOCAZIONE	Performance PRESTAZIONE
Processor	Access time
Internal (main)	Cycle time
External (secondary)	Transfer rate
Capacity CAPACITÀ	Physical Type TIPO FISICO
Word size	Semiconductor
Number of words	Magnetic
Unit of Transfer UNITÀ DI TRASFERIMENTO	Optical
Word	Magneto-Optical
Block	Physical Characteristics
Access Method METODO DI ACCESSO	Volatile/nonvolatile
Sequential	Erasable/nonerasable
Random	Organization ORGANIZZAZIONE
Associative	

COME SI USANO QUESTE CARATTERISTICHE

- $\rightarrow$ PROGETTAZIONE DI SISTEMA $\Rightarrow$ SI SCEGLIONO I LIVELLI DI CACHE (DIMENSIONE, ASSOCIATIVITÀ) IN BASE AL CYCLE TIME DELLA CPU E AL PATTERNO DI ACCESSO DEL SOFTWARE
- $\rightarrow$ DIMENSIONAMENTO $\Rightarrow$ LA WORD SIZE DEVE CORRISPONDERE AL BUS DATI DELLA CPU; IL NUMERO DI PAROLE SODDISFA IL WORKING SET DELL'APPLICAZIONE
- $\rightarrow$ OTTIMIZZAZIONE $\Rightarrow$ IL TRANSFER RATE E IL BLOCK SIZE INFLUENZANO LE PRESTAZIONI DI I/O E DI CACHING

COLLOCAZIONE DELLA MEMORIA

- INTERNA AL PROCESSORE
 - $\rightarrow$ REGISTRI: PICCOLE PORZIONI DI MEMORIA INTEGRATE DIRETTAMENTE NELL'ALU O NEL DATAPATH DELLA CPU. VANTAGGIO: ACCESSO ULTRA-VELOCE (POCHI CICLI DI CLOCK). SVANTAGGIO: COSTOSI IN TERMINI DI SILICIO
- INTERNA (MA ESTERNA DAL CORE)
 - $\rightarrow$ CACHE: MEMORIA SRAM VELOCE POSIZIONATA TRA I REGISTRI E LA RAM PRINCIPALE.
- ESTERNA (SECONDARIA)
 - $\rightarrow$ DISPOSITIVI DI STORAGE ACCESSIBILI VIA CANALI DI I/O: UNITÀ A STATO SOLIDO (SSD), HARD DISK (HDD), UNITÀ OTTICHE (CD-ROM, DVD), NASTRI MAGNETICI
 - $\rightarrow$ VANTAGGIO: CAPACITÀ MOLTO ELEVATA
 - $\rightarrow$ SVANTAGGIO: LATENZE MOLTO PIÙ ALTE E THROUGHPUT MINORE RISPETTO ALLA RAM

CAPACITÀ DELLA MEMORIA

- MEMORIA INTERNA
 - $\rightarrow$ È QUASI SEMPRE ORGANIZZATA SU PAROLE DI PIÙ BYTE
 - $\rightarrow$ PAROLA (WORD) = INSIEME DI 8, 16, 32, 64 BIT (CIÒ È 1, 2, 4, 8 BYTE SONO VALORI COMUNI)
 - $\rightarrow$ LA WORD SIZE CONDIZIONA IL BUS DATI DELLA CPU E LA WIDTH DELLE ISTRUZIONI
- MEMORIA ESTERNA
 - $\rightarrow$ SI MISURANO IN BYTE TOTALI (ES. 500 GB, 2 TB)
 - $\rightarrow$ SPESSO LE PERIFERICHE FORNISCONO ANCHE UN CONCETTO DI BLOCK O SECTOR, TIPICAMENTE 512 B O 4 KB

Perché queste differenze importano

La dimensione della parola determina quanti bit la CPU può leggere/scrivere in un singolo accesso.

In memoria interna, allineare i dati su word migliora il throughput e semplifica il datapath.

In storage esterno, organizzare grosse quantità di byte in blocchi riduce l'overhead di controllo e accelera i trasferimenti sequenziali.

Così, la collocazione fisica (registri, cache, RAM, disco) e la struttura logica (word size, block size) sono i due aspetti chiave per capire prestazioni, costi e corretto utilizzo di ogni livello della gerarchia di memoria

CARATTERISTICHE DEL TRASFERIMENTO DATI

- MEMORIA INTERNA (CACHE, RAM)
 - $\rightarrow$ UNITÀ DI TRASFERIMENTO = NUMERO DI BIT LETI O SCRITTI IN UN SINGOLO ACCESSO, DETERMINATA DALLA LARGHEZZA DEL BUS DATI
 - $\rightarrow$ PAROLA (WORD) = UNITÀ "NATURALE" DI RAPPRESENTAZIONE, TIPICAMENTE LUNGA QUANTO LA DIMENSIONE DEL REGISTRO CPU (8, 16, 32 O 64 BIT)
 - $\rightarrow$ UNITÀ INDIVISIBILE = SPESSO COINCIDE CON LA PAROLA, MA ALCUNI SISTEMI PERMETTONO L'INDIVISIBILITÀ AL BYTE
- MEMORIA ESTERNA (DISCHI, SSD)
 - $\rightarrow$ BLOCCHI = TRASFERIMENTO DI INTERI "BLOCCHI" DI DATI, SPESSO MOLTO PIÙ GRANDI DI UNA SINGOLA PAROLA (ES. 512 B, 4 KB, 64 KB)
 - $\rightarrow$ L'USO DI BLOCCHI RIDUCE L'OVERHEAD SULLE INTERFACCIE I/O E MIGLIORA IL THROUGHPUT IN LETTURA/SCRITTURE SEQUENZIALI

METODI DI ACCESSO

● SEQUENZIALE (ES. NASTRI MAGNETICI)

↓
SI LEGGE/SCRIVE SECONDO UN ORDINE PREFISSATO; PER RAGGIUNGERE UNA POSIZIONE OCCORRE "SCORRERE" TUTTE LE PRECEDENTI. IL TEMPO DI ACCESSO DIPENDE DA QUANTO È "VICINO" IL DATO CORRENTE

● DIRETTO (ES. HARD DISK)

↓
LE UNITÀ (BLOCCHI) HANNO UN INDIRIZZO FISICO UNICO; SI SPOSTA UN MECCANISMO (TESTINA) VICINO AL BLOCCO RICHIESTO, POI SI LEGGE IN MODO SEQUENZIALE. IL TEMPO È DATO DA SEEK + LATENZA ROTAZIONALE

● CASUALE/RANDOM (ES. RAM PRINCIPALE, SRAM DELLE CACHE)

↓
OGNI INDIRIZZO MAPPA DIRETTAMENTE UNA CELLA DI MEMORIA; L'ACCESSO AL DATO RICHIEDE UN TEMPO COSTANTE, INDIPENDENTE DALLA POSIZIONE

● ASSOCIATIVO (ES. CACHE FULLY-ASSOCIATIVE, CAM)

↓
SI CERCA UN DATO PER CONTENUTO INVECE CHE PER INDIRIZZO: L'HARDWARE IN PARALLELO IL CONTENUTO DESIDERATO CON UN INSIEME DI TAG DI BLOCCHI, RESTITUENDO IL BLOCCO CHE CONTIENE QUEL DATO

PRESTAZIONI

● TEMPO DI ACCESSO (ACCESS TIME)

- DEFINIZIONE: INTERVALLO FRA IL MOMENTO IN CUI PRESENTI UN INDIRIZZO ALLA MEMORIA E QUELLO IN CUI IL DATO CORRISPONDENTE È RESO DISPONIBILE SUL BUS (PER UNA LETTURA) O MEMORIZZATO (PER UNA SCRITTURA)
- MEMORIE NON CASUALI (ES. HDD): INCLUDE IL TEMPO MECCANICO DI POSIZIONAMENTO DELLA TESTINA SUL SETTORE GIUSTO (SEEK + LATENZA ROTAZIONALE)
- MEMORIE CASUALI (ES. DRAM, SRAM): È ESSENZIALMENTE IL TEMPO DI DECODIFICA DELL'INDIRIZZO, ATTIVAZIONE DELLA CELLA E PROPAGAZIONE SUL BUS

● TEMPO DI CICLO (CYCLE TIME)

- DEFINIZIONE: TEMPO MINIMO FRA DUE ACCESSI SUCCESSIVI ALLA STESSA MEMORIA $\text{TEMPO DI CICLO} = \text{TEMPO DI ACCESSO} + \text{TEMPO DI "SET-UP" O RIGENERAZIONE}$

● VELOCITÀ DI TRASFERIMENTO (TRANSFER RATE)

- DEFINIZIONE: VELOCITÀ EFFETTIVA CON CUI I DATI SCORRONO FRA LA MEMORIA ED IL BUS/PROCESSORE (BPS O BYTE/S)
- PER ACCESSO CASUALE: IL THROUGHPUT DI PICCO TENDE A $\frac{1}{\text{TEMPO DI CICLO}}$ PAROLE AL SECONDO, MOLTIPLICATO PER IL NUMERO DI BIT PER PAROLA
- PER ACCESSO NON CASUALE (BLOCCHI): QUANDO TRASFERISCI N BIT CONSECUTIVI, IL TEMPO MEDIO PER COMPLETARE L'INTERA OPERAZIONE È $T_n = T_a + \frac{N}{R}$

$T_n \Rightarrow$ TEMPO DI ACCESSO MEDIO (LATENZA)

$R \Rightarrow$ "RATE" DI TRASFERIMENTO (IN BIT PER SECONDO, BPS)

$T_n \Rightarrow$ TEMPO MEDIO PER LEGGERE/SCRIVERE N BIT

$N \Rightarrow$ NUMERO DI BIT

$\frac{N}{R} \Rightarrow$ TEMPO NECESSARIO A SPOSTARE DAVVERO N BIT UNA VOLTA CHE LA MEMORIA È PRONTA

ESEMPIO

IMMAGINA UN SSD CON $T_n = 100 \mu s$

$R = 500 \text{ MB/s} \approx 4000 \cdot 10^6 \text{ bit/s}$ $N = 4096 \text{ byte} = 32768 \text{ bit}$

$T_n = 100 \cdot 10^{-6} s + \frac{32768}{4 \cdot 10^9 s} = 0,0001 s + 0,000082 s = 108,2 \mu s$

TIP DI MEMORIA

→ SEMICONDUTTORI ("STATO SOLIDO")

RAM (VOLATILE, RISCIVIBILE, DINAMICA O STATICA) ROM (NON VOLATILE, PROGRAMMATA IN FABBRICA)

→ TECNOLOGIA MAGNETICA

HARD DISK (PACCHI MAGNETICI, TESTINE VOLANTI) TAPE (NASTRI, USATI PER BACKUP OFFLINE)

→ TECNOLOGIA OTICA

CD/DVD/BLU-RAY (LASER CHE SCRIVE/BRUCIA "PIST" E "LANDS")

→ MAGNETO-OTICO/MAGNETO-SEMICONDUTTORE

DISCHI "IBRIDI" CHE UNISCONO SCRITTURA MAGNETICA E SENSORI A SEMICONDUTTORE. SSD (SOLID-STATE DRIVE): MEMORIE FLASH NAND SU CHIP

→ TENDENZA: SPOSTAMENTO VERSO IL SOLIDO-STATE (SSD, MEMORIA FLASH), GRAZIE A LATENZE RIDOTTE, MAGGIORE ROBUSTEZZA (ASSENZA DI PARTI MECCANICHE) E CONSUMI INFERIORI

CARATTERISTICHE FISICHE

- VOLATILITÀ: RICHIEDE ALIMENTAZIONE COSTANTE PER MANTENERE I DATI (ES. DRAM, SRAM)
- NON VOLATILE: CONSERVA L'INFORMAZIONE ANCHE AD ALIMENTAZIONE SPENTA (ES. ROM, FLASH NAND (SSD), HARD DISK, CD/DVD)
- CANCELLABILITÀ/RISCIVIBILITÀ. NON CANCELLABILE: SCRIVIBILE UNA SOLA VOLTA (ONE-TIME PROGRAMMABLE) O NON SCRIVIBILE AFFATTO
- CANCELLABILE SOLO IN BLOCCO: EPROM
- RISCIVIBILE LIBERAMENTE: SI PUÒ SCRIVERE E RISCIVIERE BIT SINGOLI O PAROLE (ES. RAM, SRAM)
- CONSUMO DI POTENZA
- SEMICONDUTTORI VOLATILI: CONSUMO COSTANTE
- NON VOLATILE: CONSUMO SOLO IN SCRITTURA/LETTURA, PRATICAMENTE ZERO A RIPOSO

GERARCHIA DI MEMORIA

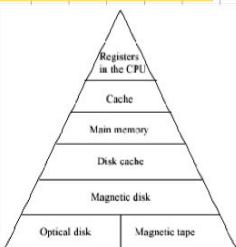
OBIETTIVI DELLA GERARCHIA DI MEMORIA

UNA GERARCHIA A PIÙ LIVELLI (REGISTRI $\rightarrow$ CACHE $\rightarrow$ RAM $\rightarrow$ DISCO $\rightarrow$ NASTRO) È UN COMPROMESSO PER CONIUGARE:

- COSTO PER BIT $\downarrow \Rightarrow$ MAN MANO CHE SCENDIAMO NELLA GERARCHIA (PIÙ "LONTANO" DALLA CPU) IL COSTO DI CIASCUN BIT DIVENTA SEMPRE PIÙ BASSO
- CAPACITÀ $\uparrow \Rightarrow$ I LIVELLI BASSI OFFRONO CAPACITÀ ENORMEMENTE MAGGIORI (1TB DI HDD) RISPETTO AI LIVELLI ALTI (CACHE L1)
- TEMPO DI ACCESSO $\uparrow \Rightarrow$ LA LATENZA CRESCE DA POCHI CICLI DI CLOCK NEI REGISTRI A MILLISECONDI SUI DISCHI E DECINE DI SECONDI SUI NASTRI
- FREQUENZA DI ACCESSO $\downarrow \Rightarrow$ IL PROCESSORE DEVE FARE MOLTI ACCESSI AI LIVELLI RAPIDI E POCHI A QUELLI LENTI, ALTRIMENTI L'INTERO SISTEMA DIVENTEREBBE TROPPO LENTO

● PRINCIPIO DI LOCALITÀ: L'ULTIMO PUNTO (DIMINUZIONE DELLA FREQUENZA DI ACCESSO AI LIVELLI BASSI) SIA SODDISFATTO, CI AFFIDIAMO AL PRINCIPIO DI LOCALITÀ:

- LOCALITÀ TEMPORALE: SE UN DATO VIENE USATO IN UN ISTANCE, È PROBABILE CHE VENGA RIUTILIZZATO A BREVE
- LOCALITÀ SPAZIALE: I DATI "VICINI" AD UN INDIRIZZO APPENA LETTO/SCRITTO SONO PROBABILI CANDIDATI PER FUTURE RICHIESTE



PRINCIPIO DI LOCALITÀ

IL PRINCIPIO DI LOCALITÀ È LA BASE CHE RENDE EFFICACI LE CACHE E, PIÙ IN GENERALE, L'INTERA GERARCHIA DI MEMORIA. SI ARTICOLA IN DUE FORME PRINCIPALI:

- **LOCALITÀ SPAZIALE** ⇒ QUANDO LA CPU ACCESSE A UN INDIRIZZO DI MEMORIA, È MOLTO PROBABILE CHE I PROSSIMI ACCESSI CADANO IN LOCALITÀ CONTIGUE (O "VICINE") A QUEL DATO BLOCCO
- FLUSSO SEQUENZIALE DELLE ISTRUZIONI: NORMALTENTE LE ISTRUZIONI SONO CARICATE UNA DOPO L'ALTRA, QUINDI LA PROSSIMA ISTRUZIONE STA "VICINO" NELL'ISTRUZIONE CACHE
- ACCESSO A VETTORI E MATRICI: I DATI SONO SPESSO ORGANIZZATI IN STRUTTURE SEQUENZIALI, PER CUI LEGGERE UN ELEMENTO SPONSTA LA FINESTRA VERSO I VICINI
- **LOCALITÀ TEMPORALE** ⇒ SE UN DATO (ISTRUZIONE O PAROLA) VIENE USATO IN UN DATO ISTANCE, È PROBABILE CHE VENGA RIUSATO A BREVE
- LOOP E CICLI: LE STESS E ISTRUZIONI VENGONO ESEGUITE RIPETUTAMENTE ANZICHÈ UNA SOLA VOLTA
- PROCEDURE: DOPO AVER CHIAMATO UNA FUNZIONE, SI TORNA A RIUSARNE LO STESSO CODICE E GLI STESSI DATI

- **CLUSTER DI LOCALITÀ** → SI RAGGRUPPANO INSIEME IN CACHE "LINEE" O "BLOCCHI" DI PIÙ PAROLE (ES. 64 B). QUANDO SERVE UN DATO, SI CARICA NON SOLO QUEL BYTE/WORD, MA L'INTERO BLOCCO CONTIGUO, SFRUTTANDO LA LOCALITÀ SPAZIALE
- **CACHE HIT RATE** → GRAZIE ALLA LOCALITÀ TEMPORALE, LE STESS E LINEE RIMANGONO "CALDE" IN CACHE E VENGONO RIUTILIZZATE PIÙ VOLTE, RIDUCENDO GLI ACCESSI ALLA RAM LENTA
- **SLORRIMENTO DELLA FINESTRA** → MAN MANO CHE IL PROGRAMMA AVANZA, LE LINEE NON PIÙ USATE VENGONO SOSTITUITE DA NUOVE "VICINE" AI DATI CORRENTI, MANTENENDO IN CACHE I CLUSTER (GRUPPI DI LOCALITÀ) CON PIÙ ALTA PROBABILITÀ DI RIUSO

IMPLICATIONI PER LA GERARCHIA DI MEMORIA

- Perché funziona
- Esecuzione sequenziale delle istruzioni → località spaziale
- Pochi livelli di procedure annidate → istruzioni vicine rimangono utili
- Ripetizione di cicli → località temporale
- Strutture dati consecutive → accessi in blocchi contigui
- Sfruttando queste proprietà, cache e buffer riescono a tenere di solito in memoria rapida i dati e le istruzioni più "probabili", massimizzando le prestazioni complessive del sistema

LOCALITÀ SPAZIALE E TEMPORALE

In letteratura si distingue a volte fra:

- Località Spaziale: è probabile che i dati/istruzioni che miserviranno siano contigui a quelli che sto usando
- Località Temporale: è probabile che i dati/istruzioni che miserviranno siano già in uso

ESEMPIO

Memoria a 2 livelli di gerarchia: M1e M2
Caso tipico che vedremo: Cache + Memoria Primaria
Livello 1 contiene 1000 parole ed ha tempo di accesso pari a 0.1 micro secondi (Es. memoria "cache")
Livello 2 contiene 100.000 parole ed ha tempo di accesso pari a 1 micro secondo (Es. memoria primaria)
Le parole vengono sempre "cercate" prima al Livello 1, se nonle si trova si passa al Livello 2
Tutte le informazioni presenti in M1 sono anche in M2, ma non viceversa (tipicamente capacità M1 << M2)
M1 è più veloce e quindi più costosa di M2

Quando l'informazione cercata non viene trovata in M1, allora viene presa da M2, ed inoltre si copia in M1 un "blocco" di informazione contenente l'informazione cercata e quelle contigue
➢ L'operazione di ricopiare in M1 un "blocco" contenente l'informazione cercata, e non trovata; serve per sfruttare il Principio di Località
Supponiamo ora che il 95% delle volte troviamo le parole nella memoria di Livello 1.
Allora il tempo di accesso medio ad una parola si può scrivere come: $0.95 \cdot 0.1 \text{ mus} + 0.05 \cdot (0.1 \text{ mus} + 1 \text{ mus}) = 0.15 \text{ mus}$

PER CALCOLARE IL TEMPO DI ACCESSO MEDIO IN UN SISTEMA A DUE LIVELLI (CACHE M1 E RAM M2) CON

- $T_1 = 0.1 \mu s \rightarrow$ TEMPO DI ACCESSO A M1
 - $T_2 = 1 \mu s \rightarrow$ TEMPO DI ACCESSO A M2
 - $h = 0.95 \rightarrow$ HIT RATE IN M1 (95%)
 - $m = 1 - h = 1 - 0.95 = 0.05 \rightarrow$ MISS RATE IN M1 (5%)
- VEDIAMO DUE SCENARI:
- 1) HIT IN M1 (PROB. 95%): BASTA IL TEMPO T_1
 - 2) MISS IN M1 (PROB. 5%): DEVO PRIMA FARE IL PROBE A M1 (T_1), POI ANDARE A LEGGERE DA M2 (T_2) E RICOPRIRE IL BLOCCO IN M1. SE TRANSCORRITO IL TEMPO DI COPIA, IL RITARDO È $T_1 + T_2$

L'ACCESS TIME MEDIO T_{avg} È $\rightarrow T_{avg} = h \cdot T_1 + m \cdot (T_1 + T_2) = 0.95 \cdot 0.1 \mu s + 0.05 \cdot (0.1 + 1) \mu s = 0.095 + 0.055 = 0.150 \mu s$

TEMPO DI ACCESSO GERARCHIA A DUE LIVELLI

Tempo di Accesso Gerarchia a due Livelli

E' facile capire che, in generale, il tempo medio di accesso ad un sistema di memoria a due livelli si può scrivere come:

$T_s = H T_1 + (1 - H) (T_1 + T_2) = T_1 + (1 - H) T_2$

T_1 :tempo di accesso memoria di primo livello

T_2 :tempo di accesso memoria di secondo livello

H : Hit ratio (percentuale di volte che il dato cercato è trovato nella memoria M_1 di primo livello)

N.B. Questa sarà la formula che useremo per il calcolo del tempo di accesso alla gerarchia cache-memoria primaria

Prestazioni Sistema Gerarchico a due livelli

- Hit ratio: percentuale degli accessi al sistema per cui la parola cercata viene trovata nella memoria di Livello 1
- T_1 : tempo di accesso alla memoria di Livello 1
- T_2 : tempo di accesso alla memoria di Livello 2

